

JARVIS SETUP BLUEPRINT

Dein eigener sprechender Assistent auf Basis von Claude Code

Ein Prompt-Blueprint · Version 1.0

von Yves Stannek

Worum es hier geht

Dieses Dokument ist kein Programm, das du installierst — es ist eine Sammlung von Prompts und ein präziser Ablauf, mit denen Claude Code dir in einer einzigen Sitzung einen lokalen, sprechenden Assistenten baut: eigene Stimme, eigenes Gedächtnis (eine Notizen-Ablage, die du selbst führst), und angebunden an dein eigenes Claude-Abo — keine zusätzliche API-Rechnung pro Nachricht.

Der zugrunde liegende Code ist ein öffentliches, kostenloses Open-Source-Projekt. Du bekommst hier nicht den Code verkauft, sondern den fertigen, getesteten Weg dorthin — inklusive der Windows-spezifischen Stolperfallen, die beim ersten eigenen Aufbau aufgetreten sind und hier bereits gelöst sind.

Was du bekommst

Den exakten Einstiegs-Prompt für eine frische Claude-Code-Sitzung · einen Windows-Kompatibilitäts-Zusatz-Prompt (falls nötig) · die Fish-Audio-Stimmen-Einrichtung Schritt für Schritt · Anpassungs-Hinweise (eigenes Gedächtnis, eigene Persönlichkeit, eigene Sprache) · eine ehrliche Sicherheitsabwägung, bevor du es startest.

Was du brauchst

1. Ein laufendes Claude-Code-Abo (die Desktop-App oder das CLI-Tool) — der Assistent läuft über deine eigene Anmeldung, nicht über eine zusätzliche API.
2. Python 3.9 oder neuer auf deinem Rechner.
3. Einen kostenlosen oder bezahlten Account bei Fish Audio (für die Stimme — optional, ohne Key spricht der Browser mit seiner eigenen, robotischen Stimme).
4. Etwa 20–30 Minuten für die erste Einrichtung.

Schritt 1 — Der Einstiegs-Prompt

Öffne eine frische Claude-Code-Sitzung in einem leeren Ordner und füge genau diesen Prompt ein. Claude Code klonet das Projekt, liest die mitgelieferte Setup-Anleitung im Repository selbst und führt dich durch den Rest.

```
Clone https://github.com/Itsme23476/jarvis-claude-code und richte es fuer mich ein. Lies zuerst SETUP-PROMPT.md im Repo und folge dem, was dort steht -- da stehen die Regeln, auf die es ankommt.
```

```
Kurzfassung: Es laeuft ueber mein eigenes Claude-Abo, also nie --bare verwenden und nie ANTHROPIC_API_KEY setzen. Frag mich nach meinem Fish-Audio-Key und trag ihn in .env ein, ohne ihn danach nochmal auszugeben. Hilf mir eine Stimme auszuwaehlen. Starte es danach und pruefe, dass es wirklich spricht, bevor du mir sagst, dass es funktioniert.
```

Warum das so und nicht anders formuliert ist

Der Hinweis auf „nie --bare, nie ANTHROPIC_API_KEY“ ist kein Kleingedrucktes — ohne ihn versucht Claude Code manchmal, sich mit einem separaten, kostenpflichtigen API-Zugang zu verbinden, statt deine bestehende Anmeldung zu nutzen. Der Prompt verhindert das von Anfang an.

Falls Claude Code nach dem Fish-Audio-Key fragt

Einen kostenlosen Account gibt es unter fish.audio. Der Key liegt danach im Account-Bereich unter „API Keys“. Du gibst ihn direkt in der Sitzung ein — er landet ausschließlich lokal in der Datei .env auf deinem eigenen Rechner und wird nirgendwo hochgeladen.

Ohne Key funktioniert der Assistent trotzdem — er spricht dann nur mit der robotischen Standard-Stimme des Browsers statt mit einer natürlichen Stimme.

Schritt 2 — Falls du auf Windows bist

Das Original-Projekt ist primär für Mac/Linux geschrieben. Auf Windows treten typischerweise drei Probleme auf: das lokale Spracherkennungs-Tool erwartet ein Kommandozeilen-Programm, das es unter Windows so nicht gibt; die Konsole stolpert über bestimmte Sonderzeichen im Start-Banner; und das Start-Skript sucht nach einem Befehl, der unter Windows anders heißt.

Reiche diesen Zusatz-Prompt direkt danach ein, wenn beim Start Fehler auftreten oder du von vornherein weißt, dass du auf Windows bist:

```
Ich bin auf Windows. Bitte pruefe und behebe folgende drei Punkte, bevor wir
starten:
```

1. Lokale Spracherkennung: Falls der Code ein externes Kommandozeilen-Tool fuer Speech-to-Text erwartet (z.B. ueber ein Homebrew-Paket), ersetze das durch die reine Python-Bibliothek 'faster-whisper' (pip install faster-whisper) -- die braucht kein externes Programm und laeuft direkt aus dem Python-Prozess heraus.
2. Konsolen-Kodierung: Falls das Start-Banner mit UnicodeEncodeError abstuerzt, stelle sicher, dass stdout/stderr beim Start explizit auf UTF-8 umgestellt werden.
3. Start-Skript: Erstelle zusaetzlich eine start.bat, die einfach 'python server.py' aufruft, damit ich das Projekt auch per Doppelklick starten kann, ohne eine Bash-Shell zu brauchen.

Warum als Prompt und nicht als fertige Datei

Diese drei Anpassungen sind bewusst als Anweisung formuliert, nicht als fertiger Code zum Einfügen. Claude Code passt sie dann direkt an den aktuellen Stand des Original-Repositories an — falls sich dort in der Zwischenzeit etwas geändert hat, funktioniert der Prompt trotzdem zuverlässig weiter, während ein starrer Code-Patch das nicht garantieren könnte.

Schritt 3 — Sicherheit verstehen, bevor du startest

Wichtig, bitte wirklich lesen

Der Assistent läuft standardmäßig im Modus „bypassPermissions“ — er kann verbundene Werkzeuge (Mail, Kalender, Ablage, Web-Suche, Shell-Befehle) OHNE Bestätigung ausführen, weil ein im Hintergrund laufender Prozess dir keine Bestätigungs-Abfrage anzeigen kann. Der Vorteil: nichts blockiert unbemerkt. Der Nachteil: ein falsch verstandenes Sprachkommando kann eine echte, nicht rückgängig zu machende Aktion auslösen — eine gelöschte Datei, eine verschickte Mail.

Für den Alltag ohne ständige Bestätigungsklicks ist das der richtige Modus. Willst du lieber auf Nummer sicher gehen, trage in der Datei .env stattdessen diese Zeile ein — dann werden Dateiänderungen ausgeführt, aber alles darüber hinaus (Mail, Shell-Befehle) bleibt gesperrt:

```
JARVIS_PERMISSION=acceptEdits
```

Grundsätzlich gilt: Dein Claude-Abo ist für dich persönlich. Diesen Assistenten auf einem eigenen Server laufen zu lassen, auf den nur du zugreifst, ist in Ordnung. Ihn so einzurichten, dass fremde Personen ihn über dein Abo nutzen können, ist es nicht — jede Person, die ihn nutzt, richtet sich am besten ihre eigene, eigenständige Installation mit ihrem eigenen Claude-Zugang ein.

Schritt 4 — Anpassen auf dich

Das Gedächtnis (Vault)

Der Assistent liest eine Ablage aus Markdown-Notizen und bezieht Antworten daraus — das mitgelieferte Beispiel ist eine Demo-Agentur. Zeig ihm stattdessen deine echte Notizen-Ablage (etwa ein bestehendes Obsidian-Vault), indem du in der `.env` diese Zeile setzt:

```
JARVIS_VAULT=~/.Dokumente/MeinVault
```

Jede Markdown-Datei mit einem kurzen Kopf-Block wird automatisch zu einem Knoten im Gedächtnis-Netz:

```
---
type: projekt
aktualisiert: 2026-09-22
---

# Mein Projekt

Kurzbeschreibung mit [[Verweisen]] auf andere Notizen.
```

Die Stimme

Stimmen-IDs lassen sich direkt bei Fish Audio durchsuchen und in der `.env` eintragen. Ein Neustart genügt, damit die neue Stimme aktiv wird — der Wert wird beim Start einmal gelesen.

Die Sprache und Persönlichkeit

Bitte Claude Code direkt in der Sitzung, den System-Prompt in `commands.py` anzupassen — zum Beispiel Tonfall, bevorzugte Antwortlänge oder feste Regeln wie „antworte immer in der Sprache, in der ich gerade geschrieben habe“.

Checkliste

- ☐ Claude Code läuft und ist bei deinem eigenen Abo angemeldet
- ☐ Einstiegs-Prompt eingefügt, Repository geklont
- ☐ Auf Windows: Zusatz-Prompt eingefügt, Start funktioniert ohne Fehler
- ☐ Fish-Audio-Key eingetragen (optional) und eine Stimme ausgewählt
- ☐ Assistent gestartet und hörbar getestet, bevor er als „fertig“ gilt
- ☐ Berechtigungsmodus bewusst gewählt (bypassPermissions oder acceptEdits)
- ☐ Eigene Notizen-Ablage verknüpft statt der Beispiel-Daten

Zum Schluss

Der größte Fehler beim ersten Aufbau ist Ungeduld beim letzten Satz des Einstiegs-Prompts: „und prüfe, dass es wirklich spricht, bevor du mir sagst, dass es funktioniert“. Ohne diesen Satz meldet Claude Code gerne „fertig“, obwohl nur der Server läuft, aber kein Ton kommt. Der Satz bleibt bewusst im Prompt — lass ihn auch bei dir stehen.